

Amador Espinoza Omar Manuel.

1.1.1.1.

¿Cómo funciona un programa de computadora?

Un programa hace que una computadora sea ejecutable, pero para esto es necesario instruir a la computadora.

1.1.1.2. ¿Lenguajes naturales frente a leng. de programación

Un lenguaje es un medio (y una herramienta) para expresar y registrar pensamientos, un conjunto de comandos concisos se llama lista de instrucciones.

1.1.1.3. ¿Qué compone un lenguaje?

Un alfabeto: símbolos utilizados para formar palabras.

Un léxico: Conjunto de palabras que el lenguaje ofrece a usuarios.

Una sintaxis: Conjunto de reglas utilizadas para crear palabras.

Una semántica: Conjunto de reglas, si una frase tiene sentido o no.

1.1.1.4. La compilación frente a la interpretación.

La programación de computadora es el acto de establecer una secuencia de palabras.

Composición: Alfabéticamente, Léxicamente, sintácticamente, semánticamente. Se puede transformar un programa de un lenguaje a de alto nivel; COMPILACIÓN, INTERPRETACIÓN.

1.1.1.5. ¿Qué hace realmente el intérprete?

Debe ser texto puro, invocar al intérprete y dejar que lea, verifica si todas las líneas están correctas, si hay error: mensaje de error, la razón se ubica antes del código.

1.1.1.6. La compilación frente a la interpretación - vent y desv.

Ventajas: Código traducido suele ser más rápido, solo el programador debe tener el compilador, el código traducido se almacena en leng. máquina.

Desventajas: La compilación puede llevar mucho tiempo, tienes que tener tantos compiladores como plataformas.

Amador Espinosa Omar Manuel.

1.1.2.1 ¿Qué es Python?

Es un lenguaje de programación de alto nivel, Python proviene de una serie.

¿Quiénes creó Python?

Nació en 1986 en Maastricht, Países Bajos por Guido van Rossum.

1.1.2.2 Una herramienta, no un reptil.

- Un proyecto de programación por hobby.

- **Objetivos de Python.** Se definió por Guido, que el lenguaje fuera fácil e intuitivo, de código abierto, que fuera comprensible, adecuado para tareas cotidianas.

1.1.2.3 ¿Qué hace especial a Python?

Que es fácil de aprender, fácil de enseñar, fácil de olvidar, fácil de entender, fácil de obtener, instalar y desplegar. (No es rápido)

1.1.2.4 ¿Rivales de Python?

Tiene oponentes como lo son Perl por Larry Wall, Ruby por Yukihiro Matsumoto, el primero más conservador que Python.

El último es más innovador y lleno de ideas nuevas.

Podemos verlo en acción, en servicios de internet, etc.

¿Por qué no Python? por programa de bajo nivel, app para dispositivos móviles.

1.1.3.1 Existe más de Python

Python 2 y Python 3, el 2 es una versión anterior al Python original. Python 3 es la versión más nueva, atravesando su evolución, creando sus estándares y hábitos. Si se comienza un proyecto nuevo en Python el más recomendable es 3.

Python alias CPython 1.1.3.2

Todos los Pythona provienen del POF en "C" o POF a menudo se denomina CPython. **Cython** - Soluciones de riesgo de Python más dolorosa por falta de eficiencia.

Jython 1.1.3.3

Python escrita en Java, es útil si desarrollas sistemas grandes.

PyPy y RPython, no se ejecuta de manera interpretativa o no tiene el C.

Python

Amador Espinoza Omar Manuel

21/09/26

1o2o11o Comenta tu viaje con pythono

Existen varias formas de tener Python3.

1o2o1o2o Comienza tu viaje con python.

descargar archivo .exe de python.

1o2o13o Comenzando tu trabajo con python.

Necesitas un editor, un consola, depurador e IDLE.

1o2o1o4 Como escribir y ejecutar primer programa

Abrir IDLE, python en File dar click en new file.

1o2o1o5 Como escribir y ejecutar primer linea.

print("Hiosss") copiarlo y pegarlo en la pestaña de IDLE

1o2o1o6 Como estropear y arreglar tu código.

Abrir el archivo nuevamente y presionar FS

1o2o1o7 Como estropear ooo

Aquí nos muestra el error. y lo arrancamos de nuevo.

1o2o1o8 Como estropear ooo.

Aquí nos muestra el error del paso anterior, donde nos muestra las líneas rojas que con rastreo, ubicación del error, contenido de la línea errónea y el nombre del error.

Felicidades acabaste el modulo 1.

Amador Espinosa Omar Manuel.

2o Fundamentos de Python.

Tipos de datos, variables, operaciones entrada y salida

2o1o1o1o Tu primer programa.

Hacer hola mundo.

2o1o1o2o Tu primer `print()`.

La palabra viene del contexto en el cual se haya utilizado la palabra.

¿De donde provienen las Funciones? `print` es es una función que viene integrada.

2o1o1o3o La función `print()`.

Una función puede tener un efecto y un resultado pero hay un tercer componente que es los argumentos las funciones en python son más versátiles.

2o1o1o4o La función `print()`

El argumento `print()` es ejemplo de cadena.

2o1o1o5o La función `print()`

`Print` junto con los parentesis y argumentos forman una invocación.

2o1o1o6o Función `print`.

Aquí realice un ejercicio `print("Omar Manuel")`

2o1o1o7o La función `print()`

El efecto es muy útil, toma los argumentos, los convierte en formato legible y envía los datos resultados al dispositivo de salida.

2o1o1o8o La función `print()` Instrucciones

La invocación es una de las muchas instrucciones en python, python requiere que no haya más de 1 instrucción por línea.

2o1o1o9o La función `print()`

Se ponemos una invocación vacía (`print()`) al imprimirla deja un hueco.

Amador Espinosa Omar Manuel.

2.01010 Los caracteres de escape y nueva línea.

La barra invertida, en cadenas, se llama caracter de escape, (se detiene por un momento).

2.01011 Los caracteres `ooo`

La barra invertida se tiene que duplicar para que no invoque error.

2.01012 Utilizando argumentos múltiples.

Las comas o argumentos separados por ellas desempeñan un papel diferente a la coma dentro de la cadena.

2.01013 La manera posicional de pasar argumentos

La manera posicional proviene del hecho de indicar por su posición.

2.01014 Los argumentos de palabra clave.

Una palabra clave identifica el argumento `end` y un signo de igual y un valor asignado.

2.01015 los argumentos de palabra clave.

Se colocas el argumento `end`. no generara nada.

2.01016 Los argumentos `ooo`

El argumento `sep` es un separador (argumento de palabra clave).

2.01017 Los argumentos `ooo`

Se colocamos `end` y `sep` colocan solo los signos colocados.

2.01018 Práctica. coloca `sep "ooo"` y `end "ooo"`

2.01019 Práctica. modificar comilla o palabra.

2.01020 Puntos clave.

`Print` es una función integrada, y las integradas siempre estan disponibles, para llamar a una función debe de utilizarse la función seguida de un paréntesis.

2.01021 Literales en Python.

Una literal se refiere a cuyos valores estan determinados por el literal mismo.

Amador Espinoza Omar Manuel

2.2.01.2 Literales en Python

Se hace uso de una cadena y número entero.

2.2.01.3 Enteros

Sistema binario, utilizan para almacenar números.

2.2.01.4 Números octales y hexadecimales.

Si un número está precedido de un 0o o 0o el número será tratado como un número octal.

2.2.01.5 Flotantes

Tienen una parte decimal no vacía, son números que pueden tener una parte fraccionaria.

2.2.01.6 Enteros frente a Flotantes

4 es un entero, 4.0 es un punto flotante.

2.2.01.7 Codificando Flotantes.

Si colocamos $6.62607E-34$, es una constante de física.

2.2.01.8 Cadenas.

Las cadenas requieren comillas o

2.2.01.9 Codificando Cadenas.

Imprimir `print("I'm Monty Python")`

2.2.01.10 Valores Booleanos

Se hace uso de verdadero y falso, denotados como 1 y 0.

2.2.01.11 Practica

2.2.01.12 Puntos Clave

Literales con notaciones para representar valores, sistema binario emplea 2 como base, las numeraciones Octales y hexadecimales emplean 8 y 16 como su base.

Amador Espinoza Omar Manuel.

2.0.10.10.

Una expresión es una combinación de valores, los operadores son símbolos especiales, los operadores aritméticos en python es (+) (-) (*) (/) etc.

Un operador unario es (-1) o (+3)

Operador binario $4 + 5$ o $12 * 3$.

Algunos operadores usan la jerarquía de prioridades, las subexpresiones dentro de parentesis se calculan primero.

2.0.4.10.11.

Una variable es una ubicación, para almacenar valores, cada variable debe tener identificador único.

Python es un tipo dinámico, también es posible utilizar operadores de asignación compuesta.

2.0.8.10.3.

Los comentarios pueden ser utilizados para info adicional, se desea colocar un comentario un #, se deben auto comentar los nombres de las variables.

2.0.6.10.12.

La función `print()` envía datos a la consola, la función puede ser utilizada para cuando termine o finalice el programa.

301010X

Los operadores de comparación se utilizan para comparar valores.

Operador	Descripción	Ejemplo
<code>==</code>	Devuelve si los valores de los operandos son iguales y False lo contrario.	<code>x == y # False</code> <code>x == x # True</code>
<code>!=</code>	Devuelve True si los valores de los operandos no son iguales.	<code>x != y # True</code> <code>x != x # False</code>
<code>></code>	Devuelve True si el valor del operando izquierdo es mayor que el derecho, False de lo contrario.	<code>x < y # True</code> <code>x < z # False</code>
<code><</code>	Devuelve True si el valor del operando izquierdo es menor que el derecho, False de lo contrario.	<code>x < y # True</code> <code>x < z # False</code>
<code>>=</code>	Devuelve True si el valor del operando izquierdo es mayor o igual al operando derecho.	<code>x >= y # False</code> <code>x >= z # True</code> <code>y >= z # True</code>
<code><=</code>	Devuelve True si el valor del operando izquierdo es menor o igual al valor del operando derecho y False de lo contrario.	

Cuando deseas ejecutar algún código solo si se cumple una condición, puedes usar sentencias condicionales (if).

<code>x = 5</code>	}	<code>x, y, z = 5, 10, 8</code>
<code>y = 10</code>		<code>print(x > z)</code>
<code>z = 8</code>		<code>print((y - 5) == x)</code>
<code>print(x > y) = False</code>		False
<code>print(y > z) = True</code>		True

3.2.1.0x.

Existen dos tipos de bucles, `while` o ejecuta una sentencia o conjunto de sentencias que una condicion booleana especifica sea verdadera.

`For` ejecuta un conjunto de sentencias muchas veces, se usa para iterar sobre una secuencia.

3.3.1x

Python es compatible con los siguientes operadores lógicos.

`and` $\rightarrow$ si ambos operandos son verdaderos, la condicion es verdadera. (`True and True`) es `True`.

`or` $\rightarrow$ si alguno de los operandos es verdadero, la condicion es verdadera. (`True or False`) es `True`.

`not` $\rightarrow$ devuelve `False` si el resultado es verdadero y devuelve `True` si es falso, `not True` es `False`.

`x = 15`, which is `0000 1111` en binario

`y = 16`, which is `0001 1000` en binario

`&` hace un bit a bit `and` (`y`), por ejemplo `x & y = 0` el cual es `0000 0000` en binario.

`|` hace un bit a bit `or`, `x | y = 31 = 0001 1111`

`~` hace un bit a bit `not` (`no`), `~x = 240 = 1111 0000`

`^` hace un bit a bit `xor`, `x ^ y = 31 = 0001 1111`

`>>` hace un desplazamiento bit a bit a la derecha `y >> 1 = 8`

`<<` hace un desplazamiento bit a bit a la izquierda `y << 3 = 1000 0000`

3.4.1.0x

La lista es un tipo de dato en Python, para almacenar múltiples objetos, es una colección ordenada y mutable. Las listas pueden indexar y actualizar, las listas pueden estar anidadas, los elementos y listas se pueden eliminar. Las listas pueden ser iteradas.

Amador Espinoza Omar Manuel

La función `len()` se puede para verificar la longitud de la lista.

3o6o1oX.

Para usar `sort()`, ordenas los elementos de una lista.

También hay método que es `reverse()`, para invertir la lista.

3o6o1oX.

Si tienes una lista `list_1` y la siguiente `list_2 = list_1`, hace que las variables `list_1` y `list_2` apunten a la misma memoria, también utilizar índices negativos. Los parámetros `start` y `end` son opcionales, se pueden eliminar rebanadas utilizando `del`. Puedes ver si los elementos existen en una lista o no utilizando `in` y `not in`.

3o7o1oX.

La comprensión de listas, te permite crear nuevas listas, de manera concisa y elegante (`for` `in` `of`). Puedes utilizar listas anidadas para crear matrices (es decir más listas bidimensionales).

Amador Espinoza Omar Manuel

4.01.01x

Una Función es un bloque de código que realiza una tarea específica cuando la función es llamada (invocada).

Cuatro tipos de funciones (básicas)

- Funciones integradas (como lo es print()).
- Módulos pre-instalados,
- Funciones definidas por el usuario
- Las funciones lambda.

Las funciones propias se pueden definir con def.

4.02.01x

Una función con parámetro.

```
def hola(name):  
    print("Hola,", name)  
hola("Greg").
```

Puedes usar argumentos.

Paso de argumentos posicionales el orden de los parámetros es relevante.

Paso de argumentos con palabras clave: en el cual el orden de los argumentos es irrelevante, primero se especifican los argumentos posicionales y después los de palabras clave.

4.03.01x

Se puede emplear una palabra clave reservada return, la instrucción return termina la función.

Se puede usar una lista con argumento de una función, una lista también puede ser un resultado de función.

4.04.01x

Una variable que existe fuera de una función tiene alcance dentro del cuerpo de la función.

Amador Espinoza Omar Manuel

```
var = 2  
def mult_by_var(x):  
    return x * var  
print(mult_by_var(7)).
```

Una variable que existe dentro de una función tiene alcance dentro del cuerpo de la función.

Se puede emplear la palabra clave reservada `global` para esto seguida de una variable.

405010X.

Una función puede invocar otras funciones incluso a sí misma. Cuando una función se invoca a sí misma, se le conoce como recursividad.

406010X.

Las Tuplas son colecciones de datos ordenadas e inmutables. Cada elemento de una tupla puede ser de un tipo diferente, y si se le agrega la coma, Python la considera una variable no tupla.

Las tuplas son inmutables, significa que no puede agregar o modificar (cambiar o quitar) elementos.

Los diccionarios son colecciones indexadas de datos, mutables y desordenadas, si se desea cambiar el valor asociado a una clave en específico, se puede hacer haciendo una referencia clave al elemento.

407010X.

Errores de sintaxis, ocurren cuando el analizador encuentra una sentencia de código que no es correcta.

Excepciones, ocurren cuando incluso una sentencia o expresión es sintéticamente correcta. Se pueden utilizar varios bloques `except` dentro de una sentencia `try`, especificar nombres de excepciones.